НИТУ «МИСИС»

Кафедра инженерной кибернетики

ОТЧЕТ

ПО

ЛАБОРАТОРНОЙ РАБОТЕ №4

«Измерение латентности кэш-памяти процессора

и оперативной памяти для заданной вычислительной системы»

учебная дисциплина «Параллельные вычисления»

Студент: Голубев И. М., БПМ-22-ПО-1

Преподаватель: Ватутин Э. И.

Оценка:

Дата защиты: 06.04.2026

2026 г.

Введение

Основная цель работы:

Познакомиться с базовыми принципами определения пропускной способности подсистемы памяти CPU

Описание решаемой задачи

Задача: Разработать программу для измерения латентности обращения к кэш-памяти различного уровня и оперативной памяти для выбранной вычислительной системы в задаче нахождения суммы элементов вектора длины N. Реализовать следующие порядки обхода элементов массива:

1. Последовательный;

2. Случайный;

3. Случайный с использованием дополнительного массива индексов.

Реализовать обработку массивов различного размера, измерить время выполнения одной итерации цикла вычисления суммы с накоплением.

Построить графики зависимости времени выполнения одной итерации цикла вычисления суммы с накоплением от объема обрабатываемых данных. На их основе сделать выводы о латентности доступа к кэш-памяти различного уровня и к оперативной памяти для выбранной вычислительной системы, полученные значения латентности представить в табличном виде. Сравнить полученные цифры с приведенными в описании лабораторной работы №4.

Входные данные:

Результат лабораторной работы — консольное приложение, на вход которого принимаются следующие параметры (соответсвенно, аргументы **pattern, max_mb, step_kb**):

1) Порядок обхода элементов массива: `seq|rnd|idx` — согласно заданию ЛР;

2) Максимальный объём обрабатываемых данных (в Мбайтах), до которого выполняются измерения;

3) Шаг изменения объёма данных (в Кбайтах).

Результат: CSV-таблица с результатами измерений: зависимость среднего времени одной итерации суммирования `ns_per_iter` от объёма обрабатываемых данных `bytes` для выбранного паттерна обхода.

Выбранные средства для разработки

Для выполнения лабораторной работы использовались следующие программные средства и технологии:

- Язык программирования: C++17
- Среда разработки: Zed
- Операционная система: Fedora Linux
- Процессор: Intel(R) Core(TM) i5-10300H CPU @ 2.50GHz
- Кэш-память CPU: L1d: 512 KiB; L1i: 512 KiB; L2: 1 MiB; L3: 8 MiB
- Размер RAM: 15Gi
- Компилятор: g++
- Сборка (оптимизированная): g++ -march=native -std=c++17 <a.cpp> -o <a>

Листинг кода программы

Код доступен в удалённом Github репозитории по ссылке:

https://github.com/g0lubew-iv/parallel-comp/blob/main/lab04/lab04.cpp

Ниже приведён полный листинг:

```
#include <cstdio>
#include <cstdint>
#include <cstring>
#include <cstdlib>
#include <chrono>

enum Pattern { SEQ, RND, IDX };

static const char* patName(Pattern p) {
    if (p == SEQ) return "sequential";
    if (p == RND) return "random_pointer_chase";
    return "random_with_index_array";
}

static Pattern parsePattern(const char* s) {
    if (strcmp(s, "seq") == 0) return SEQ;
    if (strcmp(s, "rnd") == 0) return RND;
    if (strcmp(s, "idx") == 0) return IDX;

    printf("Unknown pattern: %s (use seq|rnd|idx)\n", s);
    exit(1);
}

static const int MAX_MB = 150;
static const size_t MAX_BYTES = (size_t)MAX_MB * 1024ull * 1024ull;
static const size_t MAX_N = MAX_BYTES / sizeof(double);

alignas (64) static double   A[MAX_N];
```

```
alignas (64) static uint32_t P[MAX_N];

static volatile double sink = 0.0;

static inline uint64_t xorshift64(uint64_t& x) {
    x ^= x >> 12;
    x ^= x << 25;
    x ^= x >> 27;
    return x * 2685821657736338717ULL;
}

static void makePermutation(uint32_t* p, uint32_t n, uint64_t seed) {
    for (uint32_t i = 0; i < n; i++) p[i] = i;

    uint64_t s = seed ? seed : 1234567ULL;
    for (uint32_t i = n - 1; i > 0; i--) {
        uint32_t j = (uint32_t)(xorshift64(s) % (uint64_t)(i + 1));
        uint32_t tmp = p[i];
        p[i] = p[j];
        p[j] = tmp;
    }
}

static void permToNext(uint32_t* p, uint32_t n) {
    const uint32_t MARK = 0x80000000u;

    uint32_t first = p[0];
    for (uint32_t i = 0; i + 1 < n; i++) {
        uint32_t from = p[i];
        uint32_t to   = p[i + 1];
        p[from] = (to | MARK);
    }
    uint32_t last = p[n - 1];
    p[last] = (first | MARK);

    for (uint32_t i = 0; i < n; i++) p[i] &= ~MARK;
}

static void touchData(const double* a, size_t n) {
    double s = 0.0;
    size_t step = 64 / sizeof(double);
    for (size_t i = 0; i < n; i += step) s += a[i];
    sink = s;
}

static size_t chooseRepeats(size_t n) {
    double target_ms = 200.0;
    double est_ns = (double)n * 5.0; // грубо
    size_t r = (size_t)((target_ms * 1e6) / est_ns);
    if (r < 1) r = 1;
    if (r > 2000) r = 2000;
    return r;
}

static double measureNsPerIter(Pattern pat, size_t n, size_t repeats) {
    using clk = std::chrono::steady_clock;

    auto t0 = clk::now();
    double total = 0.0;

    if (pat == SEQ) {
        for (size_t r = 0; r < repeats; r++) {
```

```
            double s = 0.0;
            for (size_t i = 0; i < n; i++) s += A[i];
            total += s;
        }
    } else if (pat == IDX) {
        for (size_t r = 0; r < repeats; r++) {
            double s = 0.0;
            for (size_t i = 0; i < n; i++) s += A[P[i]];
            total += s;
        }
    } else { // RND pointer chase
        for (size_t r = 0; r < repeats; r++) {
            double s = 0.0;
            uint32_t cur = 0;
            for (size_t i = 0; i < n; i++) {
                cur = P[cur];
                s += A[cur];
            }
            total += s;
        }
    }

    auto t1 = clk::now();
    sink = total;

    std::chrono::duration<double> dt = t1 - t0;
    double ns_total = dt.count() * 1e9;
    return ns_total / ((double)n * (double)repeats);
}

int main(int argc, char** argv) {
    Pattern pat = parsePattern(argv[1]);
    double max_mb = atof(argv[2]);
    double step_kb = atof(argv[3]);

    size_t max_bytes = (size_t)(max_mb * 1024.0 * 1024.0);
    size_t step_bytes = (size_t)(step_kb * 1024.0);

    printf("pattern,bytes,n_elems,repeats,ns_per_iter\n");

    for (size_t bytes = step_bytes; bytes <= max_bytes; bytes += step_bytes) {
        size_t n = bytes / sizeof(double);
        if (n < 2) { continue; }

        for (size_t i = 0; i < n; i++) {
            A[i] = (double)(i % 1024) * 0.25 + 1.0;
        }
        if (pat == IDX || pat == RND) {
            makePermutation(P, (uint32_t)n, 123456789ULL ^ (uint64_t)bytes);
            if (pat == RND) permToNext(P, (uint32_t)n);
        }
        touchData(A, n);

        (void) measureNsPerIter(pat, n, 2);
        size_t repeats = chooseRepeats(n);
        double ns = measureNsPerIter(pat, n, repeats);

        printf("%s,%zu,%zu,%zu,%.6f\n", patName(pat), bytes, n, repeats, ns);
    }

    return 0;
}
```

Результаты

1) Графики **Sequential**

В широком диапазоне размеров (от сотен КБ до десятков/сотен МБ) `ns_per_iter` остаётся низким и меняется слабо (в основном, 2–3 нс/элемент на больших объёмах, с отдельными выбросами).

Явных “ступенек” на границах 32 KiB / 256 KiB / 8 MiB почти нет или они выражены очень слабо, поскольку последовательный обход отлично попадает под аппаратную предвыборку (hardware prefetch) и потоковую загрузку кэш-линий.

Так что реальная латентность кэша/DRAM скрывается конвейером, и измерение отражает в основном пропускную способность подсистемы памяти и эффективность векторизации/оптимизаций.

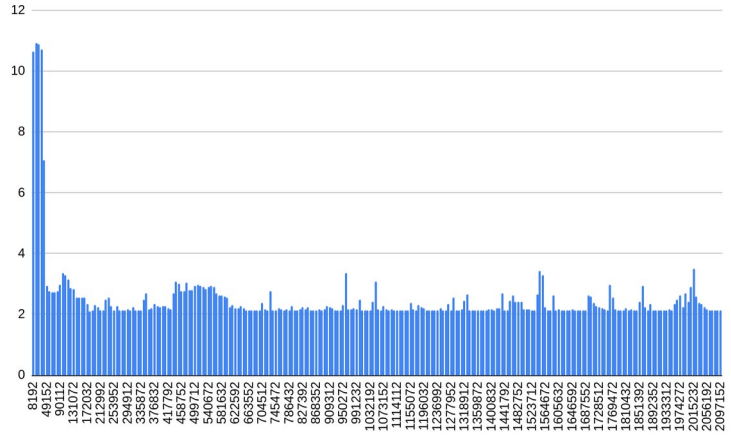

Рис. 1. Паттерн: seq; максимальный объём: 2 Мбайт; шаг: 8 Кбайт

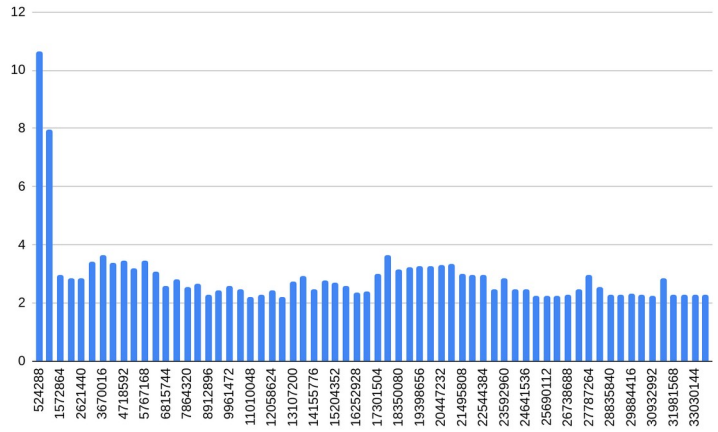

Рис. 2. Паттерн: seq; максимальный объём: 32 Мбайт; шаг: 512 Кбайт

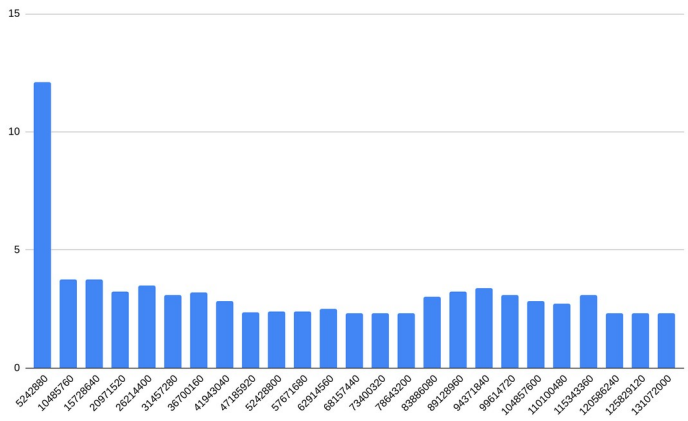

Рис. 3. Паттерн: seq; максимальный объём: 128 Мбайт; шаг: 5120 Кбайт

При последовательном доступе система памяти обеспечивает почти постоянное время обработки одного элемента даже при росте объёма данных; переходы между уровнями памяти практически не ухудшают `ns_per_iter`.

Этот график демонстрирует преимущество локальности и предвыборки и не подходит для прямой оценки латентностей уровней кэша/DRAM.

2) Графики **Random pointer chase**

При увеличении объёма данных `ns_per_iter` растёт ступенчато: появляются участки с разным “уровнем” задержки. На малых размерах до сотен КБ видны области, соответствующие работе в кэше (L2/L3).

В районе ≈ 8 MiB наблюдается выраженный скачок вверх — переход от кэша последнего уровня (L3) к DRAM. После выхода за L3 (после 8 MiB) кривая выходит на плато порядка ~70–80 нс/итерация.

Pointer chasing создаёт жёсткую зависимость по данным: следующий адрес известен только после загрузки текущего, из-за этого почти не работает перекрытие задержек, плохо помогает предвыборка. После превышения объёма L3 кэш уже не удерживает рабочий набор, так что обращения идут в DRAM, где задержка на порядок выше.

График ярко подтверждает “пирамиду памяти”: при выходе за объёмы кэшей время одного доступа резко растёт.

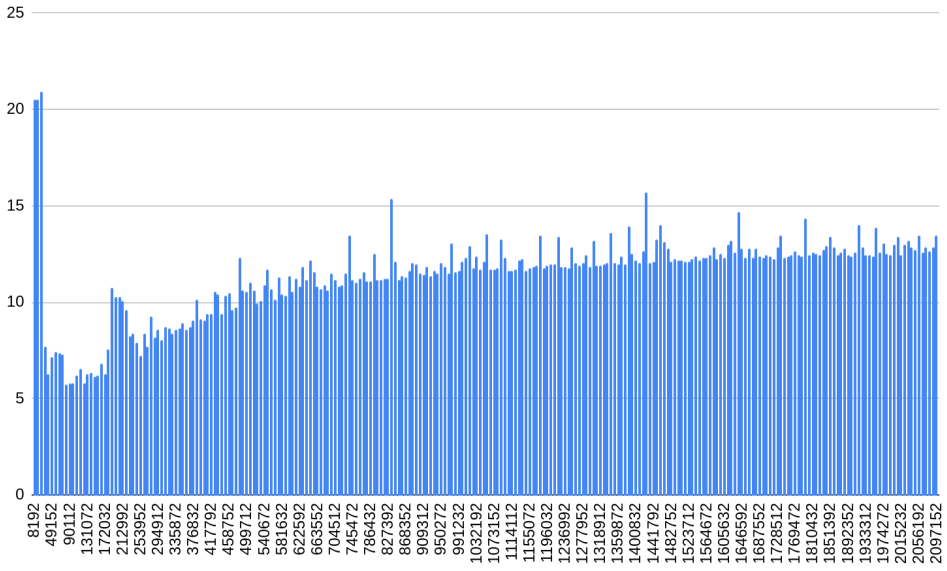

Рис. 4. Паттерн: rnd; максимальный объём: 2 Мбайт; шаг: 8 Кбайт

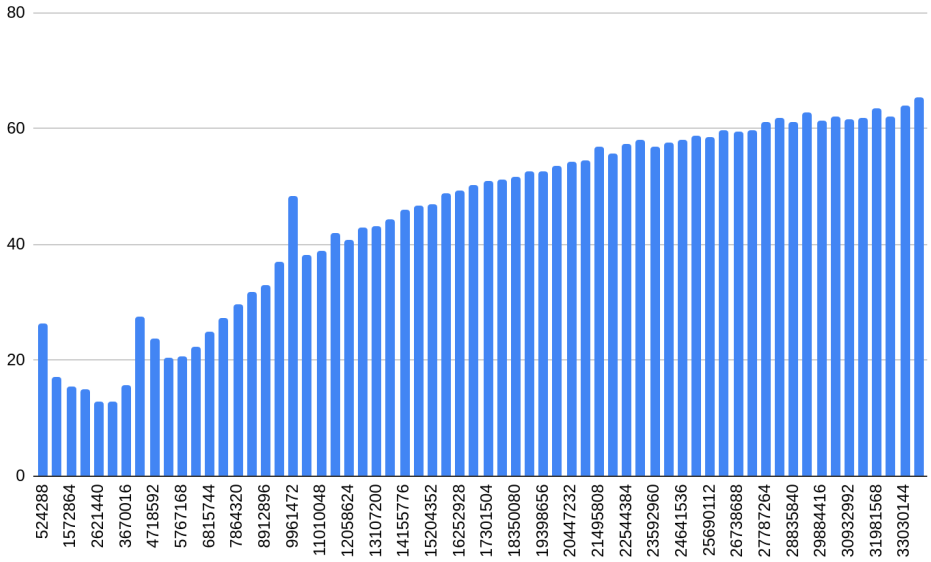

Рис. 5. Паттерн: rnd; максимальный объём: 32 Мбайт; шаг: 512 Кбайт

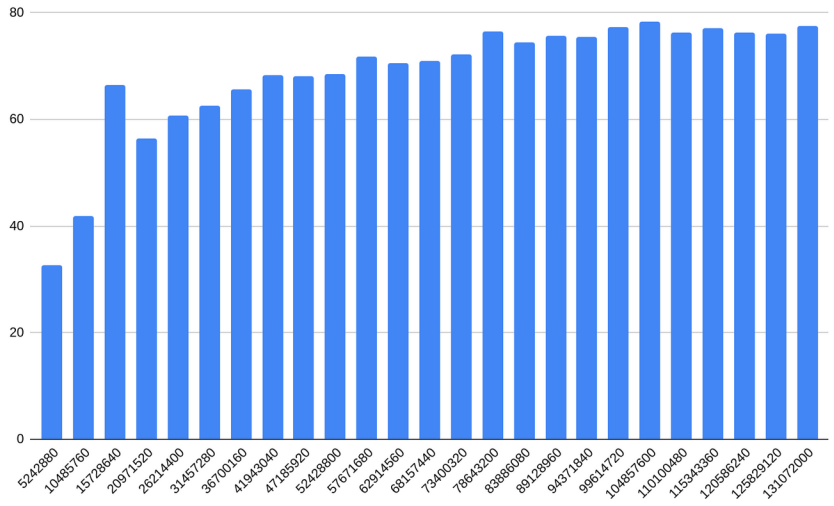

Рис. 6. Паттерн: rnd; максимальный объём: 128 Мбайт; шаг: 5120 Кбайт

Для CPU, на котором выполнялись вычисления:

- L2: ~6–7 нс (из малого диапазона),
- L3: ~12–13 нс (на нескольких МБ, до выхода в DRAM),
- DRAM: ~70–80 нс (устойчивое плато после 8 MiB).

Так что данные графики наиболее информативны для оценки латентностей уровней памяти.

3) Графики **Random with index array**

`ns_per_iter` растёт с объёмом, но заметно медленнее, чем у pointer chasing. После выхода за L3 и перехода в DRAM кривая обычно выходит на более низкое “плато”, чем у pointer chase (около 20–23 нс/итерация, тогда как pointer chase даёт 70–80 нс). Поскольку вариант a[idx[i]] даёт случайный порядок, но нет строгой зависимости “следующий адрес = результат предыдущей загрузки”, то CPU может:

- частично распараллеливать обращения,
- переупорядочивать,
- лучше скрывать латентность (Memory-Level Parallelism, OoO execution).

К тому же, индексный массив сам создаёт дополнительный поток обращений, что может слегка менять картину, но в целом задержка лучше перекрывается.

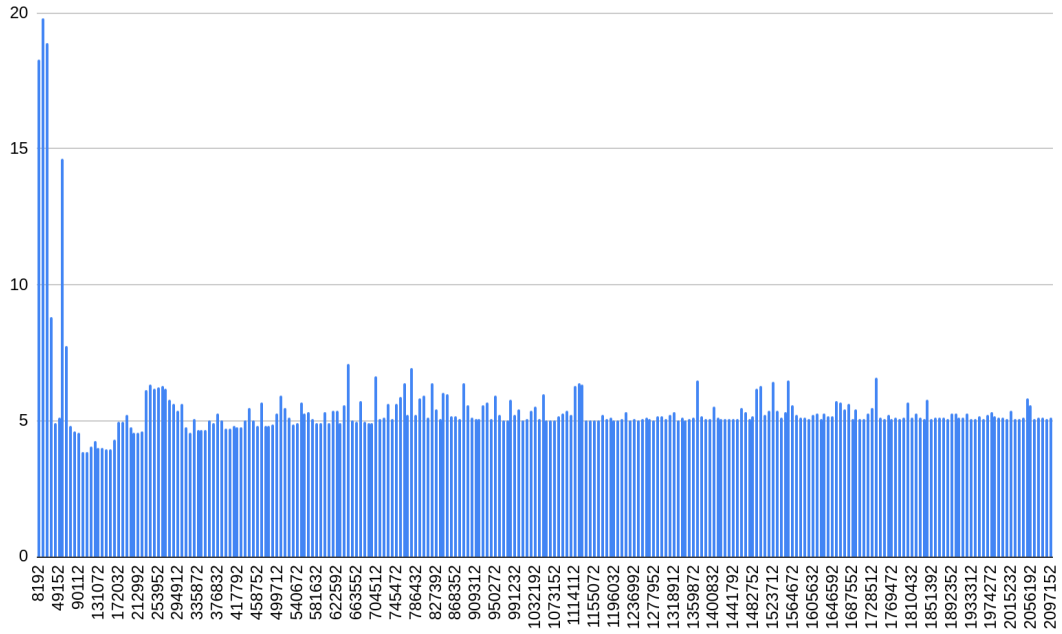

Рис. 7. Паттерн: idx; максимальный объём: 2 Мбайт; шаг: 8 Кбайт

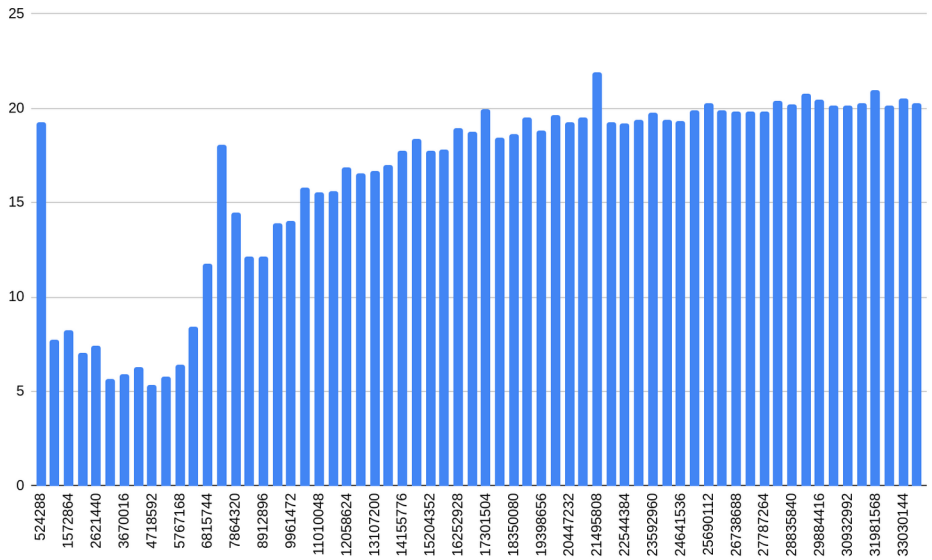

Рис. 8. Паттерн: idx; максимальный объём: 32 Мбайт; шаг: 512 Кбайт

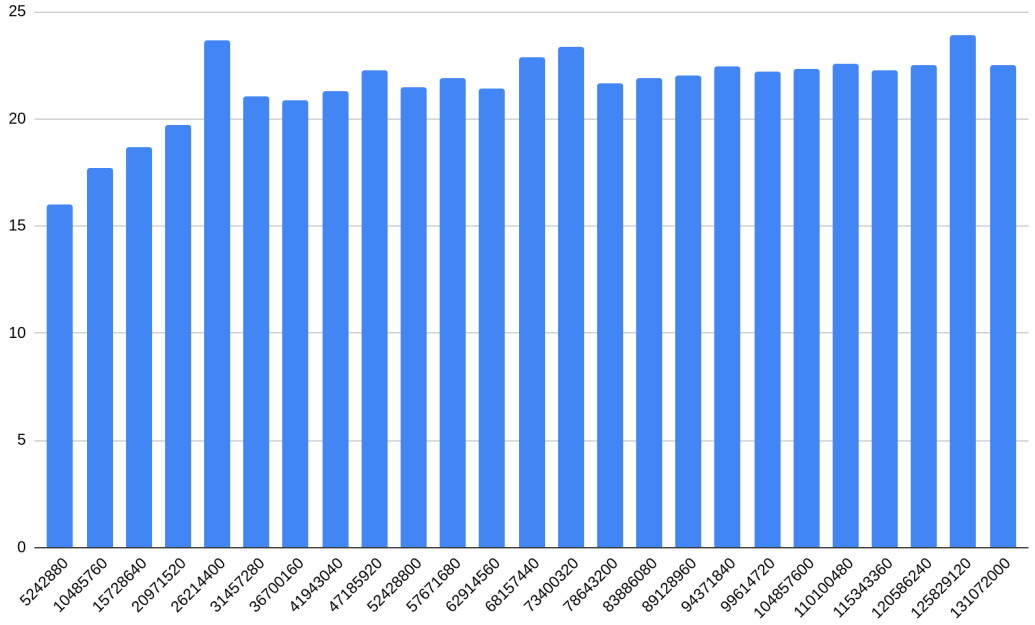

Рис. 9. Паттерн: idx; максимальный объём: 128 Мбайт; шаг: 5120 Кбайт

Таким образом, случайный доступ через индексный массив показывает, что при наличии независимых обращений процессор способен существенно скрывать задержки DRAM: время на элемент в DRAM-режиме значительно меньше, чем при pointer chasing.

Этот график демонстрирует разницу между “чистой латентностью” (pointer chase) и доступом с возможностью перекрытия задержек (index array).

Этот график демонстрирует разницу между “чистой латентностью” (pointer chase) и доступом с возможностью перекрытия задержек (index array).

Выводы

В ходе работы была разработана программа на C++, позволяющая измерять время выполнения одной итерации суммирования элементов массива при различных объёмах данных и трёх порядках обхода. На основе полученных времён построены графики зависимости `ns_per_iter` от размера обрабатываемых данных, что позволило экспериментально увидеть влияние иерархии памяти (L1/L2/L3/DRAM) на производительность и оценить характерные задержки доступа.

- **Последовательный доступ** хорошо использует локальность и предвыборку, поэтому даже при работе с данными в DRAM деградация `ns_per_iter` невелика; данный режим характеризует преимущественно пропускную способность подсистемы памяти.

- **Случайный зависимый доступ** является наиболее показательным для оценки латентности уровней памяти и наглядно демонстрирует “пирамиду памяти”: чем ниже уровень (ближе к DRAM), тем больше задержка.